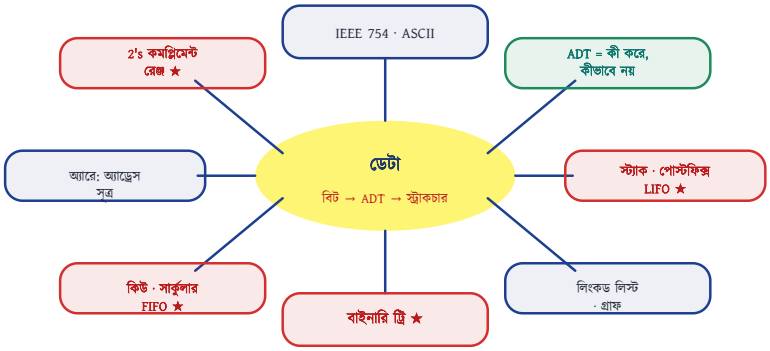
ডেটা উপস্থাপন ও অ্যাবস্ট্রাক্ট ডেটা টাইপ

বিট · 2's কমপ্লিমেন্ট · IEEE 754 · ADT · অ্যারে · লিস্ট · স্ট্যাক · কিউ · ট্রি · গ্রাফ: ১২ পাতায় পরীক্ষার প্রস্তুতি



↑ পুরো টপিক এক পাতায়: লাল = বেশি আসে, সবুজ = যে ধারণা সবকিছু জুড়ে রাখে

সিলেবাসের যে অংশ

ডেটা স্ট্রাকচার ও অ্যালগরিদমের সিলেবাস শুরু হয়: "**Internal data representation; Abstract data types; Elementary data structures: arrays, lists, stacks, queues, trees, graphs**; Advanced data structures: heaps, Fibonacci heaps, B-trees; Recursion, sorting, searching, hashing, storage management ..." **এই লেসন প্রথম তিনটি অংশ নেয়।** হিপ ও B-ট্রি, সর্টিং/সার্চিং/হ্যাশিং এবং গ্রাফ অ্যালগরিদমের আলাদা লেসন আছে।

হাই-ইল্ড তালিকা

① n-বিট 2's কমপ্লিমেন্ট: -2^{n-1} ... $2^{n-1}-1$

② -5 (8-বিট 2's) = 11111011

③ IEEE 754 সিঙ্গেল: 1 + 8 + 23, **বায়াস 127**

④ ADT = অপারেশন, ইমপ্লিমেন্টেশন নয়

⑤ রো-মেজর: $B + w[(i-L_r)n + (j-L_c)]$

⑥ স্ট্যাক = LIFO, কিউ = FIFO

⑦ সার্কুলার কিউ পূর্ণ: (rear+1) % n == front

⑧ 2 3 4 * + = 14

⑨ বাইনারি ট্রি: $n_0 = n_2 + 1$, সর্বোচ্চ $2^{h+1}-1$ নোড

⑩ অ্যাডজ. ম্যাট্রিক্স $O(V^2)$ · লিস্ট $O(V+E)$

সংকেত: **হলুদ** = মূল শব্দ · **লাল বৃত্ত** = অবশ্যই মুখস্থ · **বেগুনি** = মনে রাখার কৌশল · **লাল** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

সূচিপত্র

✓ বিট, টাইপ, সাইনড ইন্টিজার ... পৃ. ২
✓ IEEE 754, ক্যারেক্টার কোড ... পৃ. ৩
✓ অ্যাবস্ট্রাক্ট ডেটা টাইপ ... পৃ. ৪
✓ অ্যারে ও অ্যাড্রেস সূত্র ... পৃ. ৫
✓ লিংকড লিস্ট ... পৃ. ৬
✓ স্ট্যাক, ইনফিক্স → পোস্টফিক্স ... পৃ. ৭
✓ কিউ, সার্কুলার, ডেক ... পৃ. ৮
✓ ট্রি, বাইনারি ট্রি ... পৃ. ৯
✓ গ্রাফ, কমপ্লেক্সিটি টেবিল ... পৃ. ১০
✓ রাতের পরীক্ষার পাতা ... পৃ. ১১
✓ অনুশীলনী MCQ + লিখিত ... পৃ. ১২

১ বিট, ডেটা টাইপ ও সাইনড ইন্টিজার

সংজ্ঞা: **অভ্যন্তরীণ ডেটা উপস্থাপন** (Internal data representation) হলো সংখ্যা, অক্ষর ও নির্দেশনাকে **বাইনারি প্যাটার্নে** সংরক্ষণের পদ্ধতি। **বিট** = একটি 0/1, **নিবল** = 4 বিট, **বাইট** = 8 বিট, **ওয়ার্ড** = CPU-র স্বাভাবিক একক (16/32/64 বিট)। প্যাটার্নের অর্থ ঠিক করে **টাইপ**: `01000001` ইন্টিজার হিসেবে 65, ASCII হিসেবে 'A'। n বিটে 2^n **প্যাটার্ন**, তাই **আনসাইনড** n-বিট ইন্টিজারের রেঞ্জ **0 … 2^n − 1** (8-বিট 0…255, 16-বিট 0…65535)।

প্রিমিটিভ বনাম নন-প্রিমিটিভ: **প্রিমিটিভ** (Primitive): ভাষায় অন্তর্নির্মিত — int, char, float, double, boolean, pointer। **নন-প্রিমিটিভ**: এগুলো দিয়ে তৈরি — **লিনিয়ার** (অ্যারে, লিস্ট, স্ট্যাক, কিউ) ও **নন-লিনিয়ার** (ট্রি, গ্রাফ)। বেস রূপান্তর সংখ্যা-পদ্ধতির লেসনে; ১টি হেক্স অঙ্ক = 4 বিট।

C টাইপ	আকার	সাধারণ রেঞ্জ
char / unsigned char	1 B	-128…127 / 0…255
short	2 B	-32768…32767
int / long long	4 / 8 B	-2^{31}…$2^{31}-1$ / -2^{63}…
float / double	4 / 8 B	~7 / ~15–16 অঙ্ক

n বিটে ঋণাত্মক সংখ্যা রাখার তিন উপায়: **সাইন-ম্যাগনিচুড** (Sign-magnitude): MSB = চিহ্ন (0 +, 1 −), বাকিটা |x|। **1's কমপ্লিমেন্ট**: +x-এর সব বিট উল্টাও। **2's কমপ্লিমেন্ট**: সব বিট উল্টে **1 যোগ** (= 2^n − x)। CPU ব্যবহার করে **2's কমপ্লিমেন্ট**: শূন্য একটিই, আর একই অ্যাডারে যোগ-বিয়োগ দুটোই।



8 বিটে −5, তিন রূপে (4-বিটে: 1101 · 1010 · 1011)

পদ্ধতি	n-বিট রেঞ্জ	8-বিট রেঞ্জ	শূন্য	−5 (4-বিট)
সাইন-ম্যাগনিচুড	$-(2^{n-1}-1)$ … $2^{n-1}-1$	−127 … 127	**দুটি** (+0, −0)	1101
1's কমপ্লিমেন্ট	$-(2^{n-1}-1)$ … $2^{n-1}-1$	−127 … 127	**দুটি**	1010
2's কমপ্লিমেন্ট	**-2^{n-1} … $2^{n-1}-1$**	**−128 … 127**	একটি	**1011**

মনে রাখার কৌশল: **"প্রথম 1 পর্যন্ত কপি, বাকিটা উল্টাও।"** ডান দিক থেকে +x পড়ো, প্রথম 1 সহ সেখান পর্যন্ত বিট হুবহু রাখো, বাকিগুলো উল্টাও: 0000010**1** → 1111101**1**। **ডিকোড** করতে MSB-এর ওজন **-2^{n-1}**: 11111011 = −128 + 123 = **−5**।

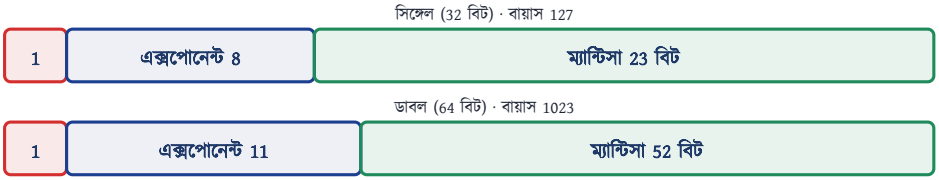
পরীক্ষার ফাঁদ: 2's কমপ্লিমেন্ট **অসম**: 8 বিটে −128 আছে, +128 নেই। "−5-এর 2's কমপ্লিমেন্ট" প্রশ্নে **10000101** (S-M) ও **11111010** (1's) হলো ভুল বিকল্প। বিট বাড়াতে **সাইন বিট কপি** করো: 1011 → 1111 1011 (সাইন এক্সটেনশন)।

ওভারফ্লো (Overflow): কেবল তখনই সম্ভব যখন **একই চিহ্নের দুটি সংখ্যা যোগ** করে ফল **বিপরীত চিহ্নের** হয় (MSB-তে ঢোকা ক্যারি ≠ বের হওয়া ক্যারি)। বিপরীত চিহ্নে **কখনো** ওভারফ্লো হয় না; MSB থেকে শুধু ক্যারি বের হলে তা ফেলে দেওয়া হয়।

সমাধানকৃত উদাহরণ ✍️ 8-বিট: 100 + 50: `01100100 + 00110010 = 10010110` → দুটি ধনাত্মক থেকে MSB 1: **ওভারফ্লো** (পড়া যায় −106)। 7 − 5 = 7 + (−5): `00000111 + 11111011 = 1 00000010` → ক্যারি বাদ → 2, ওভারফ্লো নেই।

২ IEEE 754 ফ্লোটিং পয়েন্ট ও ক্যারেক্টার কোড

সংজ্ঞা: বাস্তব সংখ্যা রাখা হয় নরমালাইজড রূপে $(-1)^S \times 1.M \times 2^{E - bias}$, তিনটি ফিল্ডে: **সাইন** S, **বায়াসড এক্সপোনেন্ট** E ও **ম্যান্টিসা** (ভগ্নাংশ) M। শুরুর 1 টি **সংরক্ষিত হয় না** (হিডেন বিট)।



S-E-M: 1 + 8 + 23 = 32 · 1 + 11 + 52 = 64

সমাধানকৃত উদাহরণ ✍️ −6.75 → সিঙ্গেল: ১. ঋণাত্মক → S = 1। ২. 6.75 = 110.11_2। ৩. নরমালাইজ: 1.1011×2^2; হিডেন 1 বাদ, M = 1011 0…0। ৪. E = 2 + 127 = 129 = **10000001**। ৫. `1 10000001 1011000…0` = **C0D80000**h। **যাচাই:** $1.1011_2 = 1.6875$; $\times 2^2 = 6.75$ ✓

ফরম্যাট	S	E	M	বায়াস	অঙ্ক
সিঙ্গেল (32)	1	8	23	**127**	~7
ডাবল (64)	1	11	52	**1023**	~15–16
E সব 0: ±0 (M = 0) বা ডিনরমাল · E সব 1: ±∞ (M = 0) বা NaN					

মনে রাখার কৌশল: **"1-8-23, 127; 1-11-52, 1023।"** k-বিট এক্সপোনেন্টে বায়াস সবসময় $2^{k-1} - 1$: $2^7 - 1 = 127$, $2^{10} - 1 = 1023$। ফিল্ডের ক্রম **S-E-M**: সাইন, এক্সপোনেন্ট, ম্যান্টিসা।

পরীক্ষার ফাঁদ: **0.1-এর নিখুঁত বাইনারি রূপ নেই** (0.000110011… পৌনঃপুনিক), তাই C, Java ও Python-এ `0.1 + 0.2 == 0.3` **মিথ্যা**। ফ্লোট তুলনা করো `|a − b| < ε` দিয়ে; টাকা রাখো পূর্ণসংখ্যায় (পয়সা)।

কোড	আকার	পরীক্ষায় যে তথ্য আসে
BCD (8421)	প্রতি দশমিক অঙ্কে 4 বিট	59 → `0101 1001`; 1010–1111 অবৈধ; 59-এর বিশুদ্ধ বাইনারি `111011`
ASCII	**7 বিট**, 128 অক্ষর	'A' = 65 (41h), 'a' = 97 (61h), '0' = 48 (30h); বড়-ছোট হাতের পার্থক্য 32
এক্সটেন্ডেড ASCII / EBCDIC	8 বিট	256 অক্ষর / IBM মেইনফ্রেম
ইউনিকোড	একটি অক্ষর-*সেট*	U+0000 … U+10FFFF; বাংলা U+0980–U+09FF; 'অ' = U+0985
UTF-8 · 16 · 32	**1–4** · 2/4 · 4 বাইট	UTF-8 ASCII-সামঞ্জস্যপূর্ণ; 'অ' = `E0 A6 85`

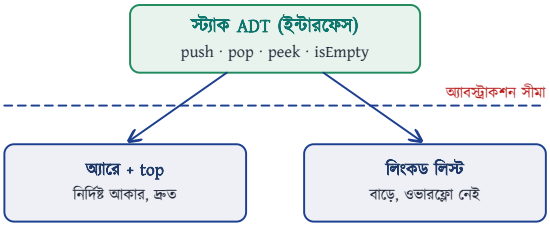
সমাধানকৃত উদাহরণ ✍️ এন্ডিয়ানেস: `0x12345678` রাখা হলো অ্যাড্রেস 100-এ। **বিগ-এন্ডিয়ান:** 100→12, 101→34, 102→56, 103→78। **লিটল-এন্ডিয়ান:** 100→**78** … 103→12। x86 লিটল-এন্ডিয়ান; **নেটওয়ার্ক বাইট অর্ডার বিগ-এন্ডিয়ান**।

পরীক্ষার ফাঁদ: **"ইউনিকোড 16-বিট"** কথাটি পুরোনো: ইউনিকোড 1 114 112 কোড পয়েন্টের সেট, আর **UTF-8/16/32 তার এনকোডিং**। ASCII **7-বিট**, 8 নয়। `int`-এর আকার **কম্পাইলার-নির্ভর** (C-তে ≥ 16 বিট)।

৩ অ্যাবস্ট্রাক্ট ডেটা টাইপ

সংজ্ঞা: **অ্যাবস্ট্রাক্ট ডেটা টাইপ (ADT)** হলো এমন একটি গাণিতিক মডেল যেখানে টাইপকে সংজ্ঞায়িত করা হয় **তার মান ও সেগুলোর ওপর অপারেশন** (ও তাদের নিয়ম) দিয়ে, **কীভাবে ইমপ্লিমেন্ট করা হলো তা দিয়ে নয়**। **ডেটা স্ট্রাকচার** হলো মেমোরিতে ডেটা সাজানোর একটি বাস্তব উপায়, যা কোনো ADT-কে ইমপ্লিমেন্ট করে। "স্ট্যাক" একটি ADT; "top ইনডেক্সসহ অ্যারে" তার একটি ডেটা স্ট্রাকচার।

> কী করে, কীভাবে নয়



এক ইন্টারফেস, অনেক ইমপ্লিমেন্টেশন

বিষয়	ADT	ডেটা স্ট্রাকচার
স্তর	লজিক্যাল, বিমূর্ত	ফিজিক্যাল, বাস্তব
বলে	**কী** করে	**কীভাবে** করে
ধারণ করে	মান + অপারেশন + নিয়ম	মেমোরি লেআউট + অ্যালগরিদম
উদাহরণ	Stack, Queue, List, Map	অ্যারে, লিংকড লিস্ট, হিপ, হ্যাশ টেবিল
কোডে	ইন্টারফেস / হেডার	ক্লাস / .c ফাইল

ADT	মূল অপারেশন	নিয়ম	যা দিয়ে তৈরি
লিস্ট	insert, delete, get(i), search	অবস্থান	অ্যারে, লিংকড লিস্ট
স্ট্যাক	push, pop, peek, isEmpty	**LIFO**	অ্যারে, লিংকড লিস্ট
কিউ	enqueue, dequeue, front, rear	**FIFO**	সার্কুলার অ্যারে, লিস্ট
ডেক (Deque)	দুই প্রান্তেই insert/delete	দুই প্রান্ত	ডাবলি লিংকড লিস্ট
প্রায়োরিটি কিউ	insert, extract-max, peek	অগ্রাধিকার	**বাইনারি হিপ**
সেট / ম্যাপ	add, contains · put(k,v), get(k)	ডুপ্লিকেট নেই	হ্যাশ টেবিল, BST
ট্রি	insert, delete, search, traverse	এক প্যারেন্ট	লিংকড নোড, অ্যারে
গ্রাফ	addVertex, addEdge, adjacent	যেকোনো থেকে যেকোনো	অ্যাডজ. ম্যাট্রিক্স / লিস্ট

ADT কেন — তিনটি সুবিধা: **এনক্যাপসুলেশন (Encapsulation)** বা তথ্য গোপন: ব্যবহারকারী শুধু অপারেশন দেখে, তাই ভেতরের রূপ বদলালেও তার কোড ভাঙে না। **পুনর্ব্যবহারযোগ্যতা (Reusability)**: একই ADT নানাভাবে ইমপ্লিমেন্ট করে বহু প্রোগ্রামে ব্যবহার করা যায়। **মডিউলারিটি (Modularity)**: প্রতিটি মডিউল আলাদাভাবে লেখা, পরীক্ষা ও ডিবাগ করা যায়। মিলে তৈরি হয় **ডেটা অ্যাবস্ট্রাকশন**, যা OOP-র ক্লাসের ভিত্তি।

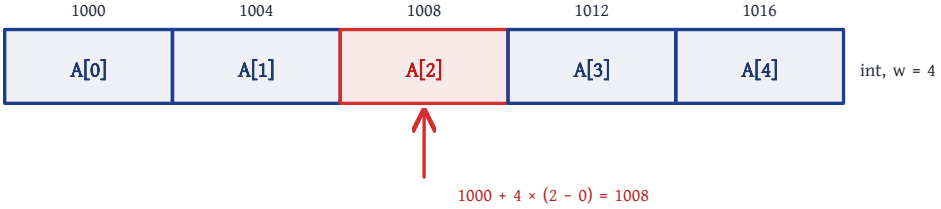
মনে রাখার কৌশল: **ADT = মেনু; ডেটা স্ট্রাকচার = রান্নাঘর।** মেনুতে লেখা কী অর্ডার করা যায় (push, pop); রান্নাঘর ঠিক করে কীভাবে রান্না হবে (অ্যারে না লিস্ট)। রান্নাঘর বদলাও, মেনু একই। সুবিধা: "E-R-M" — এনক্যাপসুলেশন, রিইউজেবিলিটি, মডিউলারিটি।

পরীক্ষার ফাঁদ: BFS ও DFS গ্রাফের ওপর চালানো **অ্যালগরিদম**, গ্রাফ ADT-র নিজস্ব অপারেশন নয়। প্রায়োরিটি কিউ **FIFO নয়**: অগ্রাধিকার অনুযায়ী সেবা দেয়। লিংকড লিস্ট একটি **ডেটা স্ট্রাকচার**; "লিস্ট" হলো ADT। "ADT সংজ্ঞায়িত করে...?" → **আচরণ**, স্টোরেজ নয়।

৪ অ্যারে ও অ্যাড্রেস নির্ণয়

সংজ্ঞা: **অ্যারে (Array)** হলো **একই টাইপের উপাদানের নির্দিষ্ট আকারের সংগ্রহ, পাশাপাশি (contiguous) মেমোরিতে**, যা ইনডেক্স দিয়ে পাওয়া যায়। উপাদানগুলো পাশাপাশি ও সমান আকারের বলে যেকোনো উপাদানের অ্যাড্রেস সূত্রে বের হয়, তাই **অ্যাক্সেস O(1)** (র‍্যান্ডম অ্যাক্সেস)। মাঝখানে ইনসার্ট বা ডিলিট করলে উপাদান সরাতে হয়: **O(n)**।

র‍্যান্ডম অ্যাক্সেস O(1)



বেস অ্যাড্রেস + প্রস্থ × অফসেট

অ্যাড্রেস সূত্র: B = বেস অ্যাড্রেস, w = প্রতি উপাদানের বাইট, L = নিম্নসীমা।

1-D: LOC(A[i]) = **B + w (i − L)** · উপাদান সংখ্যা = U − L + 1।

2-D, m সারি × n কলাম, রো-মেজর (Row-major) (C, C++, Java, Python): LOC(A[i][j]) = **B + w [(i − L_r) · n + (j − L_c)]**।

2-D, কলাম-মেজর (Column-major) (Fortran, MATLAB): LOC(A[i][j]) = **B + w [(j − L_c) · m + (i − L_r)]**।

রো-মেজরে গুণ হয় **কলাম সংখ্যা n** দিয়ে; কলাম-মেজরে **সারি সংখ্যা m** দিয়ে।

সমাধানকৃত উদাহরণ ✍ 2-D, দুই ক্রমে: `int A[20][30]` (0-ভিত্তিক, m = 20, n = 30), B = 1000, w = 4। A[5][10] বের করো।

রো-মেজর: 1000 + 4 (5 × 30 + 10) = 1000 + 4 × 160 = **1640**।

কলাম-মেজর: 1000 + 4 (10 × 20 + 5) = 1000 + 4 × 205 = **1820**।

সমাধানকৃত উদাহরণ ✍ নিম্নসীমা ≠ 0: `A[5..50]`, B = 200, w = 2। উপাদান = 50 − 5 + 1 = **46**। LOC(A[20]) = 200 + 2 (20 − 5) = **230**।

`A[−5..10]`-এ উপাদান 10 − (−5) + 1 = **16**টি।

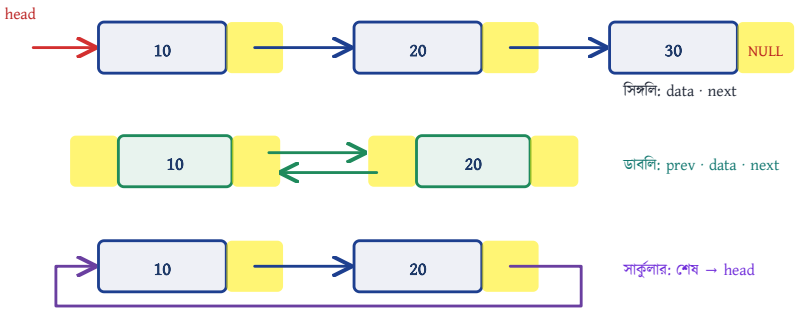
n উপাদানের অ্যারেতে অপারেশন	সময়	কেন
A[i] অ্যাক্সেস	**O(1)**	সূত্রে অ্যাড্রেস
সার্চ (অসাজানো / সাজানো)	O(n) / O(log n)	লিনিয়ার স্ক্যান / বাইনারি সার্চ
শেষে ইনসার্ট / ডিলিট	O(1)	সরাতে হয় না (জায়গা থাকলে)
k অবস্থানে ইনসার্ট / ডিলিট	**O(n)**	n − k টি উপাদান সরাতে হয়

স্পার্স ম্যাট্রিক্স (Sparse matrix): বেশিরভাগ উপাদান 0। শুধু অশূন্যগুলো রাখো **ট্রিপলেট (সারি, কলাম, মান)** হিসেবে, বা লিংকড লিস্টে। 50টি অশূন্যসহ 100 × 100 ম্যাট্রিক্সে 10 000-এর বদলে লাগে 3 × 50 = 150 ঘর। n × n **লোয়ার-ট্রায়াঙ্গুলার** ম্যাট্রিক্সে উপাদান **n(n+1)/2**।

পরীক্ষার ফাঁদ: **নিম্নসীমা** দেখো: 1-ভিত্তিক `A[1..m][1..n]` হলে আগে i ও j থেকে 1 বাদ দাও। প্রশ্নে রো-মেজর না কলাম-মেজর বলা আছে পড়ে নাও। C বাউন্ড যাচাই করে না: `int A[10]`-এ `A[10]` **অনির্ধারিত আচরণ (undefined behaviour)**, কোনো এরর বার্তা নয়।

৫ লিংকড লিস্ট: সিঙ্গলি, ডাবলি, সার্কুলার

সংজ্ঞা: **লিংকড লিস্ট** (Linked list) হলো **নোডের** ধারা; প্রতিটি নোডে থাকে ডেটা ও পরের নোডের **পয়েন্টার**, নোডগুলো মেমোরির যেকোনো জায়গায় থাকতে পারে। head পয়েন্টার প্রথম নোডে পৌঁছায়, শেষ নোডের next হলো NULL। রান-টাইমে আকার বাড়ে-কমে; র‍্যান্ডম অ্যাক্সেস নেই, তাই k-তম নোডে যেতে O(n)।



সিঙ্গলি · ডাবলি · সার্কুলার

```
struct Node { int data;
  struct Node *next; };

/* insert at head: O(1) */
struct Node *n = malloc(sizeof *n);
n->data = x;
n->next = head;
head = n;

/* delete the node after p: O(1) */
struct Node *t = p->next;
p->next = t->next;
free(t);
```

ধরন	নোড	ব্যবহার
সিঙ্গলি	data, next	স্ট্যাক, সাধারণ লিস্ট
ডাবলি	prev, data, next	ব্রাউজার back/forward, LRU ক্যাশ, ডেক
সার্কুলার	শেষ → প্রথম	**রাউন্ড-রবিন শিডিউলিং**
সার্কুলার ডাবলি	দুই লিংকই ঘোরে	মিউজিক প্লেলিস্ট, ফিবোনাচি হিপের রুট
হেডার লিস্ট	ডামি প্রথম নোড	খালি লিস্টের আলাদা কেস লাগে না

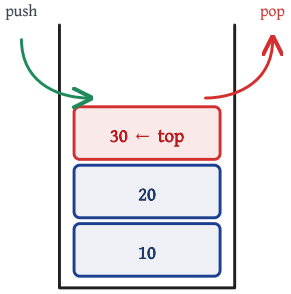
বিষয়	অ্যারে	লিংকড লিস্ট
মেমোরি	পাশাপাশি, নির্দিষ্ট আকার	ছড়ানো, রান-টাইমে বাড়ে
k-তম অ্যাক্সেস	**O(1)**	**O(n)**
সামনে ইনসার্ট/ডিলিট	O(n) (সরাতে হয়)	**O(1)**
শেষে ইনসার্ট	O(1) জায়গা থাকলে	O(n), tail পয়েন্টার থাকলে O(1)
বাড়তি জায়গা	নেই (অব্যবহৃত ঘর নষ্ট হতে পারে)	প্রতি নোডে একটি পয়েন্টার (ডাবলিতে দুটি)

মনে রাখার কৌশল: **"আগে জোড়া, পরে কাটা।"** ইনসার্টের সময় পুরোনো পয়েন্টার ঘোরানোর *আগে* নতুন নোডের next সেট করো, নইলে লিস্টের বাকিটা হারায়। মাঝের নোড: **স্লো/ফাস্ট পয়েন্টার** (ফাস্ট ২ ধাপ, স্লো ১ ধাপ)। সাইকেল: ফ্লয়েডের কচ্ছপ-খরগোশ।

পরীক্ষার ফাঁদ: **সিঙ্গলি** লিস্টে নির্দিষ্ট নোড ডিলিট করতে তার **আগের নোড** লাগে, খুঁজতে O(n); **ডাবলি** লিস্টে তা **O(1)**। সিঙ্গলি লিস্ট উল্টাতে O(n) সময় ও O(1) জায়গা (তিন পয়েন্টার: prev, curr, next)।

৬ স্ট্যাক, ইনফিক্স → পোস্টফিক্স ও পোস্টফিক্স মূল্যায়ন

সংজ্ঞা: **স্ট্যাক** (Stack) এমন লিস্ট যেখানে ইনসার্ট (push) ও ডিলিট (pop) হয় **এক প্রান্তে, top-এ: শেষে ঢুকলে আগে বের (LIFO)**। অ্যারে রূপে `top = -1` মানে খালি; `top == MAX-1` অবস্থায় push হলো **ওভারফ্লো**; `top == -1` অবস্থায় pop হলো **আন্ডারফ্লো**। লিংকড রূপে head-এ push/pop হয়, মেমোরি না ফুরালে ওভারফ্লো নেই। সব অপারেশন **O(1)**।



শেষে ঢুকলে আগে বের

```
void push(int x) {
  if (top == MAX-1) { puts("Overflow"); return; }
  s[++top] = x; }
int pop(void) {
  if (top == -1) { puts("Underflow"); return -1; }
  return s[top--]; }
```

প্রয়োগ: **ফাংশন কল ও রিকার্শন** (কল স্ট্যাকে রিটার্ন অ্যাড্রেস ও লোকাল ভেরিয়েবল) · এক্সপ্রেশন রূপান্তর ও মূল্যায়ন · ব্র্যাকেট মিলানো · undo/redo · ব্রাউজার back · ব্যাকট্র্যাকিং (গোলকধাঁধা, N-queens) · DFS · স্ট্রিং উল্টানো। যেকোনো রিকার্সিভ অ্যালগরিদম স্পষ্ট স্ট্যাক দিয়ে লেখা যায়।

ইনফিক্স → পোস্টফিক্সের নিয়ম: বাম থেকে ডানে পড়ো। **অপারেন্ড** → আউটপুট। **(** → push। **)** → "(" না আসা পর্যন্ত pop করে আউটপুট। **অপারেটর** → top-এ **বেশি বা সমান** অগ্রাধিকারের অপারেটর থাকলে pop (সমান হলে কেবল বাম-সহযোগী হলে), তারপর push। শেষে সব pop। **^ ডান-সহযোগী।**

অগ্রাধিকার: **^ > * / > + -**

পড়া	স্ট্যাক	আউটপুট
A		A
+	+	A
B	+	A B
*	+ *	A B
C	+ *	A B C
-	-	A B C * +
D, /, E	- /	A B C * + D E
শেষ		**A B C * + D E / -**

সমাধানকৃত উদাহরণ ✍ পোস্টফিক্স মূল্যায়ন: অপারেন্ড → push; অপারেটর → pop **B** (top), pop **A**, push **A op B**।

`2 3 4 * +`: push 2, 3, 4; * → 3 × 4 = 12; + → 2 + 12 = **14** (ইনফিক্সে 2 + 3 * 4)।

`5 6 2 + * 12 4 / -`: 6 + 2 = 8; 5 × 8 = 40; 12 / 4 = 3; 40 - 3 = **37**।

সমাধানকৃত উদাহরণ ✍ তিন নোটেশন: ইনফিক্স `(A + B) * (C - D)`

পোস্টফিক্স (রিভার্স পোলিশ): **A B + C D - ***

প্রিফিক্স (পোলিশ): *** + A B - C D**

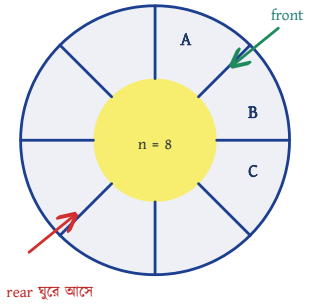
পোস্টফিক্সে **বন্ধনী বা অগ্রাধিকারের নিয়ম লাগে না**, তাই কম্পাইলার ও ক্যালকুলেটর এটি ব্যবহার করে।

মনে রাখার কৌশল: **"প্লেটের স্তূপ।"** সবশেষে ধোয়া প্লেটটিই আগে ব্যবহার হয়। পোস্টফিক্স মূল্যায়নে **দ্বিতীয় pop হলো বাম অপারেন্ড**: "pop B, pop A, হিসাব A op B" — - ও /-এর ক্ষেত্রে জরুরি।

পরীক্ষার ফাঁদ: `2 3 4 * +` = **14, 20 নয়**; 20 হতো `2 3 + 4 *`। 1, 2, 3 ক্রমে push করলে কখনো **3, 1, 2** ক্রমে pop হয় না। বেস কেসে না পৌঁছানো রিকার্শনের শেষ **স্ট্যাক ওভারফ্লো**।

৭ কিউ: লিনিয়ার, সার্কুলার, ডেক, প্রায়োরিটি

সংজ্ঞা: **কিউ** (Queue) **rear**-এ ইনসার্ট (*enqueue*) ও **front** থেকে ডিলিট (*dequeue*) করে: **আগে ঢুকলে আগে বের (FIFO)**। সাধারণ অ্যারেতে front ও rear শুধু সামনে এগোয়, তাই খালি হওয়া ঘর আর ব্যবহার হয় না (অর্ধেক খালি থেকেও "পূর্ণ"); **সার্কুলার কিউ** ইনডেক্সকে `% n` দিয়ে ঘুরিয়ে আনে।



rear = (rear + 1) % n

সার্কুলার কিউ (n আকারের অ্যারে)	শর্ত
Enqueue	`rear = (rear + 1) % n; Q[rear] = x`
Dequeue	`x = Q[front]; front = (front + 1) % n`
পূর্ণ	**(rear + 1) % n == front**
খালি	`front == -1` (এক-ঘর-খালি রূপে `front == rear`)
উপাদান সংখ্যা	`(rear - front + n) % n` (এক-ঘর-খালি রূপে)

দুই রীতি: `front = rear = -1` মানে খালি: n ঘরই ব্যবহারযোগ্য। `front = rear = 0` মানে খালি: একটি ঘর ফাঁকা থাকে, ধারণক্ষমতা **n - 1**।

সমাধানকৃত উদাহরণ ✍️ ট্রেস: খালি কিউ: enqueue(1), enqueue(2), enqueue(3) → [1, 2, 3]; dequeue() সরায় **1** → [2, 3]; enqueue(4) → [2, 3, 4]; dequeue() সরায় **2** → [3, 4]। front = **3**, rear = 4। একই push-pop **স্ট্যাকে** করলে থাকে [1, 2], top-এ 2।

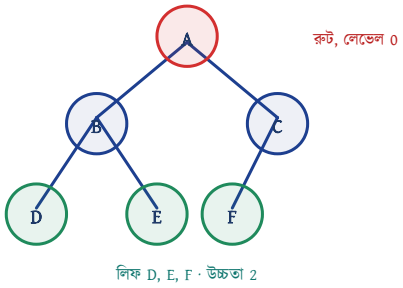
ধরন	নিয়ম	সাধারণ ব্যবহার
লিনিয়ার কিউ	FIFO, ইনডেক্স শুধু সামনে এগোয়	সাধারণ বাফার
সার্কুলার কিউ	FIFO, ইনডেক্স % n দিয়ে ঘোরে	CPU **রাউন্ড-রবিন**, কিবোর্ড ও নেটওয়ার্ক বাফার
ডেক	**দুই** প্রান্তেই ইনসার্ট ও ডিলিট	স্লাইডিং-উইন্ডো সর্বোচ্চ, undo ইতিহাস
সীমিত ডেক	ইনপুট-সীমিত: এক প্রান্তে ইনসার্ট · আউটপুট-সীমিত: এক প্রান্তে ডিলিট	—
প্রায়োরিটি কিউ	সর্বোচ্চ অগ্রাধিকার আগে; সমান হলে FIFO	ডাইকস্ট্রা, OS জব শিডিউলিং, হাফম্যান

মনে রাখার কৌশল: **"ব্যাংকের কাউন্টারের লাইন।"** পেছনে (rear) দাঁড়াও, সামনে (front) থেকে বের হও। **D**ouble-**E**nded **QUE**ue = ডেক। ব্যবহার: BFS, প্রিন্টার স্পুলিং, CPU শিডিউলিং, দ্রুত ও ধীর ডিভাইসের মাঝে বাফার।

পরীক্ষার ফাঁদ: প্রায়োরিটি কিউ সবচেয়ে ভালো **বাইনারি হিপে**: insert ও delete-max দুটোই O(log n) — অসাজানো অ্যারেতে delete O(n)। **দুটি স্ট্যাক** দিয়ে কিউ বানানো যায় (অ্যামর্টাইজড O(1))। "BFS-এ কোনটি?" → কিউ; "DFS-এ?" → স্ট্যাক।

৮ ট্রি ও বাইনারি ট্রি

সংজ্ঞা: **ট্রি** (Tree) হলো একটি **রুট**সহ সংযুক্ত, চক্রহীন স্তরবিন্যাস; বাকি প্রতিটি নোডের ঠিক একটি প্যারেন্ট, তাই n নোডে **n - 1 এজ**। **বাইনারি ট্রি:** সর্বোচ্চ দুটি চাইল্ড (বাম, ডান)। পরিভাষা: **লিফ**, **ইন্টার্নাল** নোড, **সিবলিং**, **ডিগ্রি** = চাইল্ড সংখ্যা, **লেভেল** (রুট = 0), **উচ্চতা** = রুট থেকে দীর্ঘতম লিফ পর্যন্ত এজ সংখ্যা।



F হলো C-এর **বাম** চাইল্ড

ট্রাভার্সাল	ক্রম	এই ট্রির ফল
প্রিঅর্ডার	নোড, বাম, ডান	A B D E C F
ইনঅর্ডার	বাম, নোড, ডান	D B E A F C
পোস্টঅর্ডার	বাম, ডান, নোড	**D E B F C A**
লেভেল অর্ডার	স্তরে স্তরে (কিউ)	A B C D E F

বাইনারি ট্রির ধর্ম (রুট লেভেল 0, উচ্চতা h)	সূত্র	যাচাই (h = 2)
লেভেল i-তে সর্বোচ্চ নোড	2^i	লেভেল 2: 4
উচ্চতা h-এর ট্রিতে সর্বোচ্চ নোড	$2^{h+1} - 1$	7
n নোডে ন্যূনতম উচ্চতা	$\lceil \log_2(n + 1) \rceil - 1$	n = 7 → 2
লিফ বনাম ডিগ্রি-2 নোড	$n_0 = n_2 + 1$	পারফেক্ট h = 2: 4 = 3 + 1
n নোডে ভিন্ন বাইনারি ট্রি	ক্যাটালান: $C(2n, n)/(n + 1)$	n = 3 → 5

বাইনারি ট্রির প্রকার: **ফুল / স্ট্রিক্ট**: প্রতিটি নোডের 0 বা 2 চাইল্ড। **কমপ্লিট**: শেষটি ছাড়া সব লেভেল পূর্ণ, শেষটি **বাম থেকে** ভরা (হিপের আকার)। **পারফেক্ট**: প্রতিটি ইন্টার্নাল নোডের 2 চাইল্ড, সব লিফ এক লেভেলে। **স্কিউড**: প্রতিটির এক চাইল্ড — আসলে লিস্ট।

অ্যারে উপস্থাপন: লেভেল ধরে সাজাও। **0-ভিত্তিক:** বাম = **2i + 1**, ডান = 2i + 2, প্যারেন্ট = $\lfloor (i - 1)/2 \rfloor$। **1-ভিত্তিক:** বাম = 2i, ডান = 2i + 1, প্যারেন্ট = $\lfloor i/2 \rfloor$। কমপ্লিট ট্রির জন্য আদর্শ; স্কিউডে অপচয়।

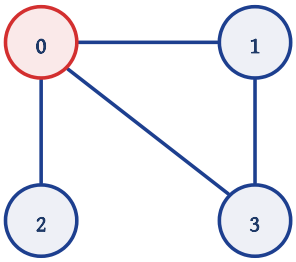
সমাধানকৃত উদাহরণ ✍️ দুই ট্রাভার্সাল থেকে ট্রি গঠন: প্রিঅর্ডার A B D E C F, ইনঅর্ডার D B E A F C। প্রিঅর্ডারের প্রথমটি রুট **A**; ইনঅর্ডারে D B E তার বাম সাবট্রি, F C ডান সাবট্রি। **B** বাম অংশের রুট (D, E); **C** ডান অংশের রুট, F তার বামে। পোস্টঅর্ডার: **D E B F C A**। ইনঅর্ডার + প্রি/পোস্ট মিলে ট্রি অনন্যভাবে নির্ধারিত; শুধু প্রি + পোস্টে নয়।

মনে রাখার কৌশল: নামই বলে **নোড কোথায় বসে**: **প্রি** = নোড আগে, **ইন** = নোড মাঝে, **পোস্ট** = নোড শেষে; বাম সবসময় ডানের আগে। **BST**: বাম < নোড < ডান, তাই এর **ইনঅর্ডার সাজানো**।

পরীক্ষার ফাঁদ: লেভেল গোনা কোথা থেকে শুরু দেখো: রুট লেভেল **1** হলে লেভেল i-তে সর্বোচ্চ 2^{i-1} ও উচ্চতা h-এ $2^h - 1$ নোড। BST সার্চ **গড়ে** O(log n), কিন্তু **স্কিউড হলে O(n)** — তাই AVL ও রেড-ব্ল্যাক ট্রি।

৯ গ্রাফ, এবং প্রতিটি অপারেশনের খরচ

সংজ্ঞা: **গ্রাফ (Graph)** G = (V, E) হলো ভার্টেক্সের সেট ও তাদের জোড়া সংযোগকারী এজের সেট। **আনডিরেক্টেড** বা **ডিরেক্টেড** (ডাইগ্রাফ), **ওয়েটেড** বা নয়। **ডিগ্রি** = এক ভার্টেক্সে এজ সংখ্যা (ডাইগ্রাফে ইন-/আউট-ডিগ্রি)। **পাথ** এজ দিয়ে ভার্টেক্স জোড়ে; **সাইকেল** শুরুতে ফেরে; প্রতিটি জোড়া পাথে যুক্ত হলে গ্রাফ **কানেক্টেড**। **ট্রি হলো সংযুক্ত চক্রহীন গ্রাফ**।



V = 4, E = 4

আনডিরেক্টেড, ৪টি এজ

·	0	1	2	3
0	0	1	1	1
1	1	0	0	1
2	1	0	0	0
3	1	1	0	0

```
0 → 1 → 2 → 3
1 → 0 → 3
2 → 0
3 → 0 → 1
```

বিষয়	অ্যাডজেসেন্সি ম্যাট্রিক্স	অ্যাডজেসেন্সি লিস্ট
জায়গা	**$O(V^2)$**	**$O(V + E)$**
(u, v) কি এজ?	**$O(1)$**	O(u-এর ডিগ্রি)
u-এর সব প্রতিবেশী	$O(V)$	O(u-এর ডিগ্রি)
উপযুক্ত	**ঘন (dense)** গ্রাফে	**বিরল (sparse)** গ্রাফে (বাস্তবের বেশিরভাগ)

গণনার তথ্য: সর্বোচ্চ এজ: আনডিরেক্টেড সিম্পল গ্রাফে **$n(n - 1)/2$**, ডিরেক্টেডে $n(n - 1)$ — তাই K_5-এ 10টি এজ। **হ্যান্ডশেকিং:** সব ডিগ্রির যোগফল = $2|E|$, তাই বিজোড়-ডিগ্রির ভার্টেক্স জোড়-সংখ্যক। n ভার্টেক্সের ট্রিতে $n - 1$ এজ; **DAG** হলো চক্রহীন ডিরেক্টেড গ্রাফ। **ইনসিডেন্স ম্যাট্রিক্স** $V \times E$।

স্ট্রাকচার	অ্যাক্সেস	সার্চ	ইনসার্ট	ডিলিট
অ্যারে (অসাজানো)	$O(1)$	$O(n)$	$O(n)$	$O(n)$
সাজানো অ্যারে	$O(1)$	$O(\log n)$	$O(n)$	$O(n)$
সিঙ্গলি লিংকড লিস্ট	$O(n)$	$O(n)$	head-এ $O(1)$	জানা নোডের পরে $O(1)$
স্ট্যাক / কিউ	top/front $O(1)$	$O(n)$	$O(1)$	$O(1)$
BST (গড় / সবচেয়ে খারাপ)	—	$O(\log n)$ / $O(n)$	$O(\log n)$ / $O(n)$	$O(\log n)$ / $O(n)$
বাইনারি হিপ	max $O(1)$	$O(n)$	$O(\log n)$	$O(\log n)$

পরীক্ষার ফাঁদ: "অ্যাডজেসেন্সি লিস্টের জায়গা" **$O(V + E)$**, $O(E)$ নয়: এজ না থাকলেও প্রতিটি ভার্টেক্সের লিস্ট-হেড লাগে। কমপ্লিট গ্রাফ ঘন, তাই সেখানে ম্যাট্রিক্স ভালো। গ্রাফে সাইকেল ও একাধিক কম্পোনেন্ট থাকতে পারে; ট্রিতে নয়।

অধ্যায়ের বাকিটা কোথায়: **হিপ, ফিবোনাচি হিপ, B-ট্রি** → অ্যাডভান্সড ডেটা স্ট্রাকচার লেসন। **রিকার্শন, সর্টিং, সার্চিং, হ্যাশিং** → পরের লেসন। **BFS, DFS, টপোলজিক্যাল সর্ট, MST, শর্টেস্ট পাথ** → গ্রাফ অ্যালগরিদম লেসন।

পরীক্ষার আগের রাতের পাতা

তথ্য-পাতা:

n বিট → 2^n প্যাটার্ন; আনসাইনড 0…2^n−1

2's: -2^{n-1} … 2^{n-1}−1 (8-বিট −128…127)

−5: S-M 10000101 · 1's 11111010

2's-এ −5 (8-বিট) = **11111011**

ওভারফ্লো: একই চিহ্ন ঢোকে, বিপরীত চিহ্ন বের হয়

ASCII 7-বিট · 'A' 65 · 'a' 97 · '0' 48

IEEE সিঙ্গেল 1-8-23, বায়াস 127

IEEE ডাবল 1-11-52, বায়াস 1023

ADT = কী (অপারেশন); DS = কীভাবে

লিটল-এন্ডিয়ান: ছোট বাইট নিচু অ্যাড্রেসে

রো-মেজর: B + w[(i−L_r)n + (j−L_c)]

কলাম-মেজর: B + w[(j−L_c)m + (i−L_r)]

স্ট্যাক LIFO · ওভারফ্লো top == MAX−1

সার্কুলার পূর্ণ: (rear+1) % n == front

লেভেল i ≤ 2^i নোড; উচ্চতা h ≤ 2^{h+1}−1

$n_0 = n_2 + 1$ · এজ = n − 1

চাইল্ড 2i+1, 2i+2 (0-ভিত্তিক)

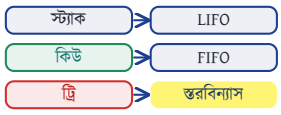
ম্যাট্রিক্স $O(V^2)$ · লিস্ট O(V+E) · সর্বোচ্চ এজ n(n−1)/2

MCQ-র সেরা ১২ এক-লাইন

1. 2's-এ শূন্য একটি; S-M ও 1's-এ দুটি
2. প্রথম 1 পর্যন্ত কপি, বাকিটা উল্টাও
3. ইউনিকোড সেট; UTF-8 হলো 1–4 বাইট
4. ফ্লোটিং পয়েন্টে 0.1 + 0.2 =/ 0.3
5. স্ট্যাক ADT: push, pop, peek, isEmpty
6. অ্যারে অ্যাক্সেস O(1); মাঝে ইনসার্ট O(n)
7. ডাবলি লিস্টে নির্দিষ্ট নোড ডিলিট O(1)
8. 2 3 4 * + = 14; A+B*C → ABC*+
9. অগ্রাধিকার ^ > * / > + −; ^ ডান-সহযোগী
10. রিকার্শনে স্ট্যাক; BFS-এ কিউ
11. প্রায়োরিটি কিউ → বাইনারি হিপ
12. BST-র ইনঅর্ডার সাজানো

প্রধান ফাঁদ: 10000101 হলো S-M, 2's নয় · −128-এর জোড়া +128 নেই · ক্যারি আউট =/ ওভারফ্লো · ASCII 7-বিট · int-এর আকার কম্পাইলার-নির্ভর · নিম্নসীমা =/ 0 · দ্বিতীয় pop বাম অপারেন্ড · প্রায়োরিটি কিউ FIFO নয় · BST-র সবচেয়ে খারাপ O(n) · লেভেল গোনা 0 না 1 থেকে।

সব কৌশল একসাথে: প্রথম 1 পর্যন্ত কপি, বাকিটা উল্টাও · 1-8-23/127, 1-11-52/1023, বায়াস 2^{k-1}−1 · ADT = মেনু, DS = রান্নাঘর · আগে জোড়া, পরে কাটা · প্লেটের স্তূপ · ব্যাংকের লাইন · প্রি/ইন/পোস্ট = নোড কোথায় বসে।



তিন রকম ক্রম

কাজ	স্ট্রাকচার	কাজ	স্ট্রাকচার
রিকার্শন, undo, পোস্টফিক্স	**স্ট্যাক**	BFS, প্রিন্টার স্পুল	**কিউ**
রাউন্ড-রবিন CPU	সার্কুলার কিউ / লিস্ট	জব অগ্রাধিকার, ডাইকস্ট্রা	প্রায়োরিটি কিউ (হিপ)

সমাধানকৃত উদাহরণ ✍️ এক মিনিটে পাঁচটি সংখ্যা: (১) −20-এর 8-বিট 2's: 00010100 → **11101100**। (২) 16-বিট 2's রেঞ্জ: **−32768 … 32767**। (৩) A[10][10], B = 500, w = 2, রো-মেজরে A[3][4]: 500 + 2 (34) = **568**। (৪) ১০টি লিফের বাইনারি ট্রিতে ডিগ্রি-2 নোড **৯**টি। (৫) ৬ ভার্টেক্সের সিম্পল আনডিরেক্টেড গ্রাফে সর্বোচ্চ **15**টি এজ।

✓ অনুশীলনী MCQ (পরীক্ষার ধাঁচে, ১০ নম্বর)

১. কোন ADT LIFO মেনে চলে?
ক) কিউ খ) স্ট্যাক
গ) প্রায়োরিটি কিউ ঘ) সার্কুলার কিউ

২. -5-এর 8-বিট 2's কমপ্লিমেন্ট:
ক) 10000101 খ) 11111010
গ) 11111011 ঘ) 00000101

৩. 8-বিট 2's কমপ্লিমেন্টের রেঞ্জ:
ক) -127 ... 127 খ) -128 ... 127
গ) 0 ... 255 ঘ) -128 ... 128

৪. IEEE 754 সিঙ্গেলের এক্সপোনেন্ট বায়াস:
ক) 127 খ) 128
গ) 1023 ঘ) 255

৫. `int A[20][30]`, B = 1000, w = 4, রো-মেজরে A[5][10]:
ক) 1820 খ) 1640
গ) 1600 ঘ) 1660

৬. পোস্টফিক্স `2 3 4 * +`-এর মান:
ক) 20 খ) 24
গ) 9 ঘ) 14

৭. n আকারের সার্কুলার কিউ পূর্ণ যখন:
ক) front == rear খ) rear == n - 1
গ) (rear+1)%n == front ঘ) front == 0

৮. enq 1, 2, 3, deq, enq 4, deq। front?
ক) 3 খ) 4
গ) 2 ঘ) 1

৯. উচ্চতা 3-এর বাইনারি ট্রিতে সর্বোচ্চ নোড:
ক) 8 খ) 7
গ) 15 ঘ) 16

১০. অ্যাডজেসেন্সি ম্যাট্রিক্সের জায়গা:
ক) O(V + E) খ) O(E)
গ) O(V log V) ঘ) $O(V^2)$

উত্তরমালা → ১-খ · ২-গ · ৩-খ · ৪-ক · ৫-খ · ৬-ঘ · ৭-গ · ৮-ক · ৯-গ · ১০-ঘ

লিখিত প্রশ্ন: অ্যাবস্ট্রাক্ট ডেটা টাইপ কী? স্ট্যাক ADT এবং ওভারফ্লো ও আন্ডারফ্লোসহ এর অ্যারে ইমপ্লিমেন্টেশন বর্ণনা করো। স্ট্যাক দেখিয়ে `A + B * C - D / E`-কে পোস্টফিক্সে রূপান্তর করো এবং `5 6 2 + * 12 4 / -` এর মান নির্ণয় করো। ১০ নম্বর

১ **ADT (২)** — মান ও অপারেশন দিয়ে সংজ্ঞায়িত টাইপ, ইমপ্লিমেন্টেশন-নিরপেক্ষ; সুবিধা: এনক্যাপসুলেশন, রিইউজেবিলিটি, মডিউলারিটি। ডেটা স্ট্রাকচার একে ইমপ্লিমেন্ট করে। ২

২ **স্ট্যাক ADT (২)** — LIFO লিস্ট; push, pop, peek, isEmpty, isFull, সব O(1)। ব্যবহার: কল স্ট্যাক, এক্সপ্রেশন, undo, ব্যাকট্র্যাকিং। ২

৩ **অ্যারে ইমপ্লিমেন্টেশন (২)** — `top = -1` খালি; push: `top == MAX-1` হলে → **ওভারফ্লো**, নইলে `s[++top] = x`; pop: `top == -1` হলে → **আন্ডারফ্লো**, নইলে `s[top--]` ফেরত। ২

৪ **ইনফিক্স → পোস্টফিক্স (২)** — A আউট; + push; B আউট; * + এর চেয়ে বড়, push; C আউট; - এলে * ও + pop (≥ অগ্রাধিকার), - push; D আউট; / push; E আউট; / ও - pop → **A B C * + D E / -**। ২

৫ **মূল্যায়ন (২)** — push 5, 6, 2; + → 8; * → 40; push 12, 4; / → 3; - → 40 - 3 = **37**। ২

পরীক্ষার আগে ✓

- ✓ -x কে S-M, 1's ও 2's-এ লিখতে পারি
- ✓ IEEE 754 সিঙ্গেল ও ডাবলের বিন্যাস জানি
- ✓ ADT সংজ্ঞা ও DS-এর সাথে পার্থক্য বলতে পারি
- ✓ 2-D-র দুই অ্যাড্রেস সূত্র প্রয়োগ করতে পারি
- ✓ ইনফিক্স → পোস্টফিক্স ও তার মান বের করতে পারি
- ✓ সার্কুলার কিউ ও ট্রির সূত্র বলতে পারি